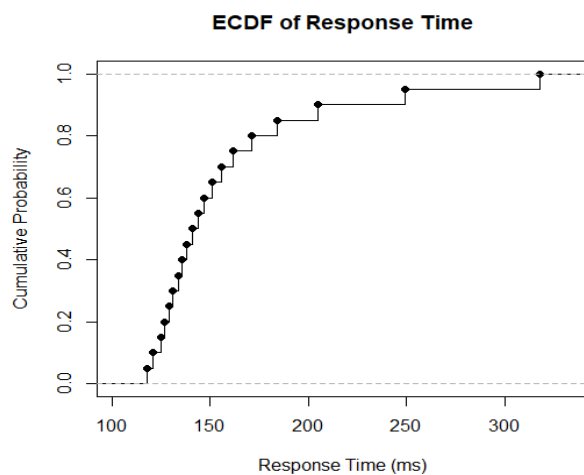


Question 1

Task 1: Construct an ECDF

```
, # Create the dataset
response_time <- c(118, 121, 125, 127, 129, 131, 134, 136, 138, 141,
                  144, 147, 151, 156, 162, 171, 184, 205, 249, 318)

# Create ECDF plot
plot(ecdf(response_time),
     main = "ECDF of Response Time",
     xlab = "Response Time (ms)",
     ylab = "Cumulative Probability",
     verticals = TRUE
     do.points = TRUE)
```

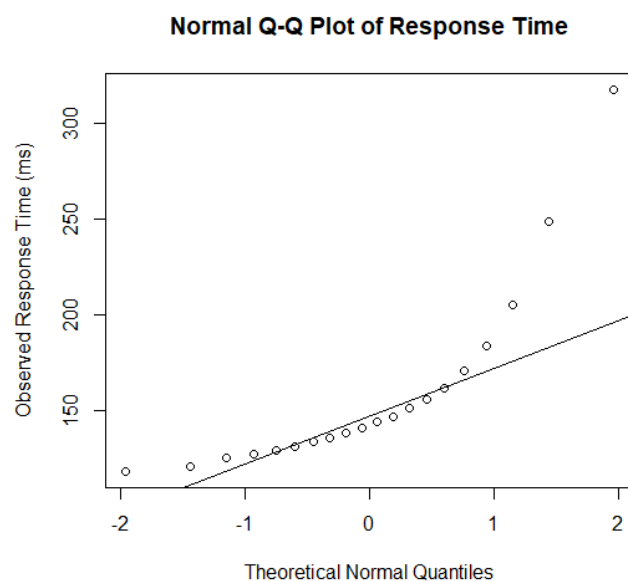


Task 2: Construct a Normal Q-Q Plot

```
# Create the dataset
response_time <- c(118, 121, 125, 127, 129, 131, 134, 136, 138, 141,
                  144, 147, 151, 156, 162, 171, 184, 205, 249, 318)

# Create Normal Q-Q Plot
qqnorm(response_time,
```

```
main = "Normal Q-Q Plot of Response Time",  
xlab = "Theoretical Normal Quantiles",  
ylab = "Observed Response Time (ms)"  
  
# Add reference line  
qqline(response_time)
```



Task 3: Evaluate the Normality Claim

```
# Create the dataset  
response_time <- c(118, 121, 125, 127, 129, 131, 134, 136, 138, 141,  
                  144, 147, 151, 156, 162, 171, 184, 205, 249, 318)  
  
# Calculate mean  
mean(response_time)  
  
# Calculate median  
median(response_time)  
  
# Calculate standard deviation  
sd(response_time)  
  
# Perform Shapiro-Wilk normality test  
shapiro.test(response_time)
```

Shapiro-Wilk normality test

data: response_time

W = 0.7351, p-value = 0.0001079

Task 4: Identify Skewness and Extreme Observations

```
# Create the dataset
```

```
response_time <- c(118, 121, 125, 127, 129, 131, 134, 136, 138, 141,  
                  144, 147, 151, 156, 162, 171, 184, 205, 249, 318)
```

```
# Calculate Quartiles
```

```
Q1 <- quantile(response_time, 0.25)
```

```
Q3 <- quantile(response_time, 0.75)
```

```
# Calculate IQR
```

```
IQR_value <- IQR(response_time)
```

```
# Calculate lower and upper limits
```

```
lower_limit <- Q1 - 1.5 * IQR_value
```

```
upper_limit <- Q3 + 1.5 * IQR_value
```

```
# Display limits
```

```
lower_limit
```

```
upper_limit
```

```
# Identify outliers
```

```
outliers <- response_time[response_time < lower_limit |  
                          response_time > upper_limit]
```

```
print("Possible Outliers:")
```

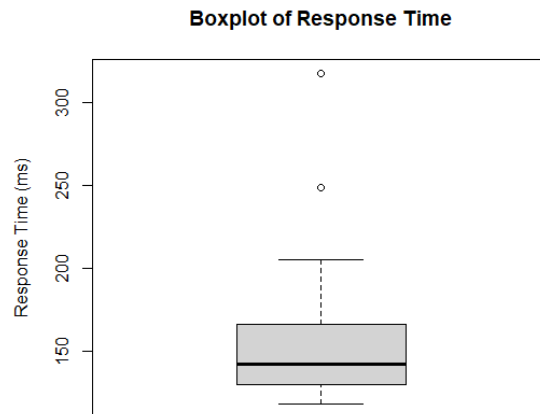
```
print(outliers)
```

```
# Create a boxplot

boxplot(response_time,

        main = "Boxplot of Response Time",

        ylab = "Response Time (ms)")
```



Task 5: Recommend Monitoring Limits

```
# Create the dataset

response_time <- c(118, 121, 125, 127, 129, 131, 134, 136, 138, 141,

                  144, 147, 151, 156, 162, 171, 184, 205, 249, 318)

# Calculate Mean

mean_value <- mean(response_time)

# Calculate Standard Deviation

sd_value <- sd(response_time)

# Calculate limits

lower_limit <- mean_value - sd_value

upper_limit <- mean_value + sd_value

# Display results

print(paste("Mean =", mean_value))
```

```
print(paste("Standard Deviation =", sd_value))
```

```
print(paste("Lower Limit =", lower_limit))
```

```
print(paste("Upper Limit =", upper_limit))
```

Mean = 159.35

Standard Deviation = 47.29

Lower Limit = 112.06

Upper Limit = 206.64

Question 2

Task 1: Compare All Four Distributions

```
# Create the data
```

```
L1 <- c(42, 44, 45, 46, 47, 48, 49, 50, 51, 53)
```

```
L2 <- c(38, 41, 44, 47, 50, 53, 56, 59, 62, 65)
```

```
L3 <- c(45, 46, 46, 47, 47, 48, 48, 49, 49, 50)
```

```
L4 <- c(40, 41, 42, 43, 44, 45, 48, 55, 61, 72)
```

```
# Combine data
```

```
cycle_time <- data.frame(
```

```
  Line = rep(c("L1", "L2", "L3", "L4"), each = 10),
```

```
  Time = c(L1, L2, L3, L4)
```

```
)
```

```
# Compare all distributions using boxplot
```

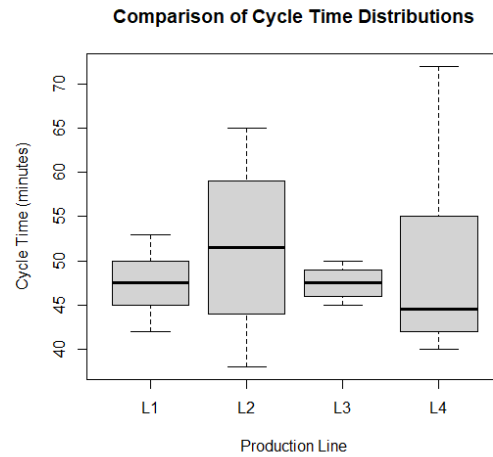
```
boxplot(Time ~ Line,
```

```
  data = cycle_time,
```

```
  main = "Comparison of Cycle Time Distributions",
```

```
  xlab = "Production Line",
```

```
  ylab = "Cycle Time (minutes)")
```



Task 2: Bean Plot / Equivalent Distribution Visualization

```
L1 <- c(42, 44, 45, 46, 47, 48, 49, 50, 51, 53)
```

```
L2 <- c(38, 41, 44, 47, 50, 53, 56, 59, 62, 65)
```

```
L3 <- c(45, 46, 46, 47, 47, 48, 48, 49, 49, 50)
```

```
L4 <- c(40, 41, 42, 43, 44, 45, 48, 55, 61, 72)
```

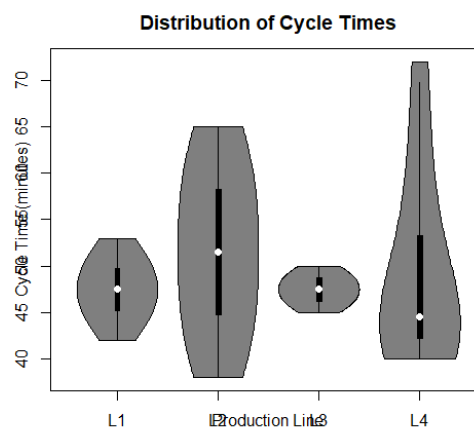
```
vioplot(L1, L2, L3, L4,
```

```
  names = c("L1", "L2", "L3", "L4"),
```

```
  main = "Distribution of Cycle Times",
```

```
  xlab = "Production Line",
```

```
  ylab = "Cycle Time (minutes)")
```



Task 3: Find Greatest Variability and Most Concentrated Distribution

```

# Create the data
L1 <- c(42, 44, 45, 46, 47, 48, 49, 50, 51, 53)
L2 <- c(38, 41, 44, 47, 50, 53, 56, 59, 62, 65)
L3 <- c(45, 46, 46, 47, 47, 48, 48, 49, 49, 50)
L4 <- c(40, 41, 42, 43, 44, 45, 48, 55, 61, 72)

# Calculate Standard Deviations
variability <- c(
  L1 = sd(L1),
  L2 = sd(L2),
  L3 = sd(L3),
  L4 = sd(L4)
)

# Display variability
print(variability)

# Find greatest variability
print(paste("Greatest Variability:", names(which.max(variability))))

# Find most concentrated distribution
print(paste("Most Concentrated:", names(which.min(variability))))

L1 = 3.43
L2 = 9.08
L3 = 1.58
L4 = 10.33

Greatest Variability: L4
Most Concentrated: L3

```

Task 4: Why Comparing Only Means Is Insufficient

```

# Calculate means
means <- c(
  L1 = mean(L1),
  L2 = mean(L2),
  L3 = mean(L3),

```

```

L4 = mean(L4)
)
print(means)
L1 = 47.5
L2 = 51.5
L3 = 47.5
L4 = 49.1

```

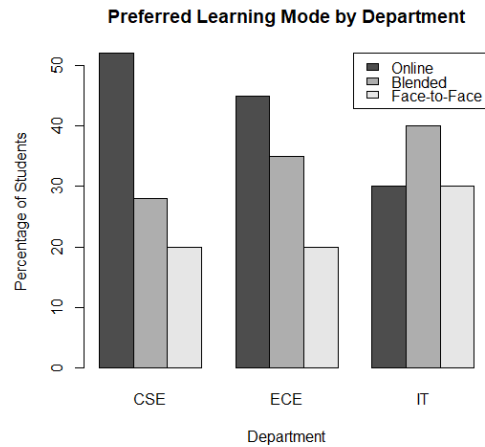
Question 3

Task 1A: Side-by-Side Bar Chart

```

# Create the data
learning_mode <- matrix(c(
  52, 28, 20,
  45, 35, 20,
  30, 40, 30
),
nrow = 3,
byrow = TRUE)
# Add row and column names
rownames(learning_mode) <- c("CSE", "ECE", "IT")
colnames(learning_mode) <- c("Online", "Blended", "Face-to-Face")
# Create side-by-side bar chart
barplot(t(learning_mode),
  beside = TRUE,
  main = "Preferred Learning Mode by Department",
  xlab = "Department",
  ylab = "Percentage of Students",
  legend.text = TRUE,
  args.legend = list(x = "topright"))

```



Task 1B: 100% Stacked Bar Chart

Create the data

```
learning_mode <- matrix(c(
```

```
  52, 28, 20,
```

```
  45, 35, 20,
```

```
  30, 40, 30
```

```
),
```

```
nrow = 3,
```

```
byrow = TRUE)
```

Add row and column names

```
rownames(learning_mode) <- c("CSE", "ECE", "IT")
```

```
colnames(learning_mode) <- c("Online", "Blended", "Face-to-Face")
```

Create 100% stacked bar chart

```
barplot(t(learning_mode),
```

```
  beside = FALSE,
```

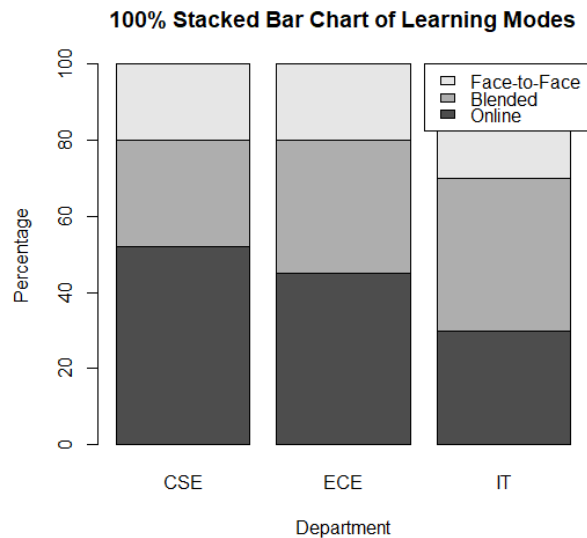
```
  main = "100% Stacked Bar Chart of Learning Modes",
```

```
  xlab = "Department",
```

```
  ylab = "Percentage",
```

```
legend.text = TRUE,

args.legend = list(x = "topright"))
```



Task 2: Which Chart Is Better?

The side-by-side bar chart is better for comparing individual proportions.

Task 3: Why Are Separate Pie Charts Less Effective?

Separate pie charts are less effective because:

- It is difficult to compare angles and areas between different pies
- Small differences are harder to identify
- Comparing the same category across departments takes more effort
- Pie charts do not provide a common baseline

For example, comparing **Online preference of 52% in CSE and 45% in ECE** is easier with bars than by comparing two pie slices.

Task 4: Design Choice That Can Exaggerate Small Differences

Online preference percentages

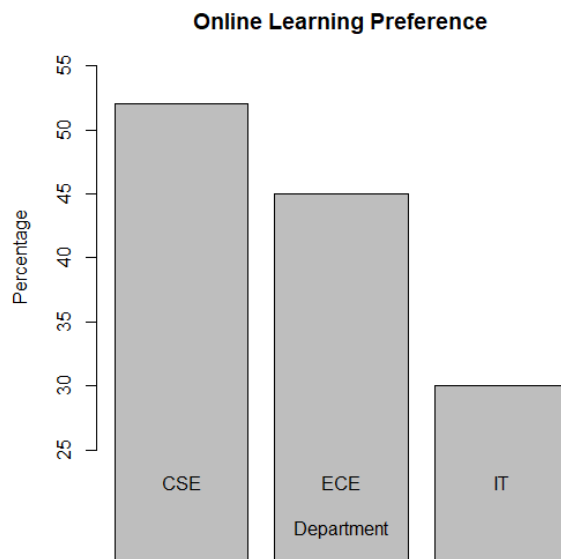
```
online <- c(52, 45, 30)
```

Create misleading bar chart using truncated Y-axis

```
barplot(online,
```

```
names.arg = c("CSE", "ECE", "IT"),
```

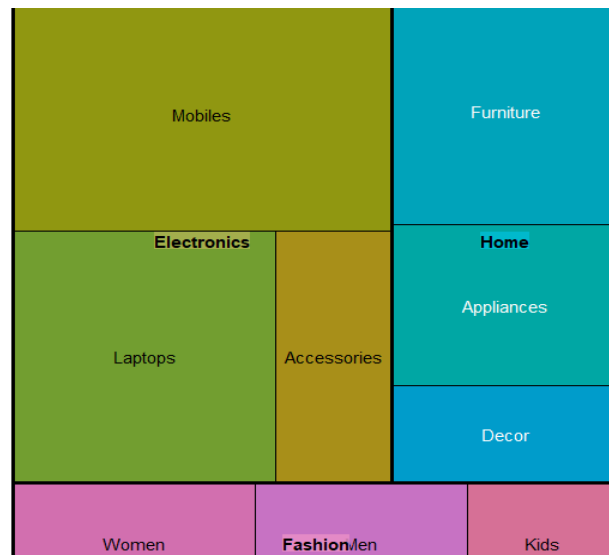
```
ylim = c(25, 55),
main = "Online Learning Preference",
xlab = "Department",
ylab = "Percentage")
```



Question 4

Task:1 Construct a treemap showing category and subcategory revenue.

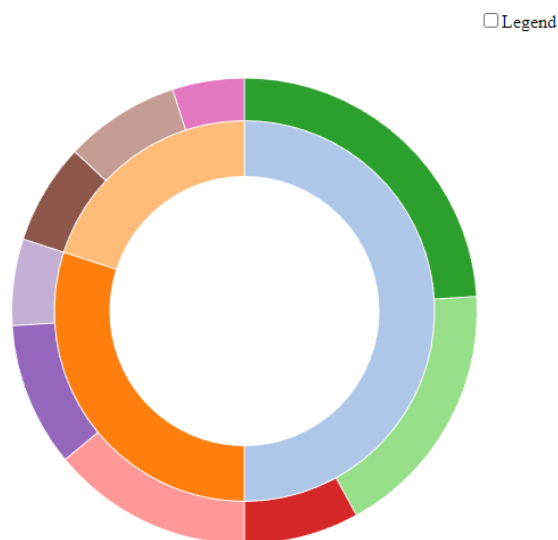
```
treemap(
  data,
  index = c("Category", "Subcategory"),
  vSize = "Revenue",
  title = "Treemap of Category and Subcategory Revenue"
)
```



Task 2: Construct a Sunburst Chart

```
sunburst_data <- data.frame(
  sequence = paste(data$Category, data$Subcategory, sep = "-"),
  size = data$Revenue
)

sunburst(sunburst_data)
```



Task 3: Find the Largest Category and Percentage

```
category_total <- aggregate(
  Revenue ~ Category,
  data = data,
```

```
FUN = sum
)
print(category_total)
```

Output:

```
Category Revenue
1 Electronics    50
2  Fashion      20
3   Home        30
```

Task 4: Compare Treemap and Sunburst

Treemap	Sunburst
Uses rectangles	Uses circular rings
Shows values using area	Shows values using circular segments
Easy to compare category sizes	Easy to understand hierarchy
Best for comparing proportions	Best for showing Category → Subcategory relationship

Task 5: Situation Where Area Encoding Can Mislead

Area encoding can mislead when the values are **very close to each other** because humans cannot accurately compare small differences in areas.

Example:

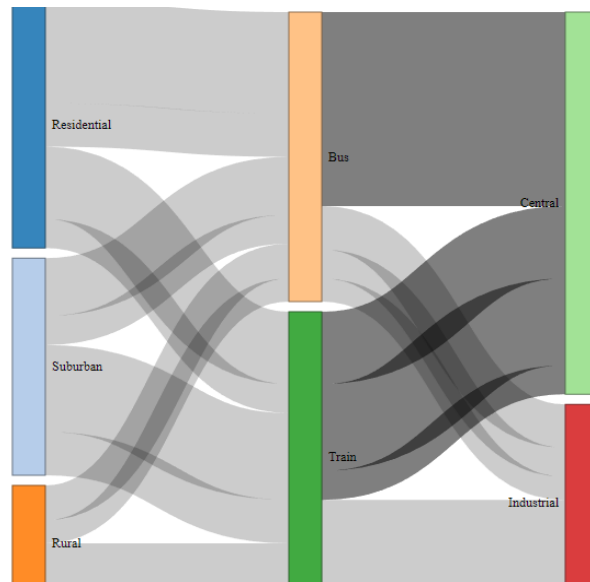
If one subcategory has ₹24 lakh revenue and another has ₹23 lakh, their areas may look almost the same, making the exact difference difficult to understand.

Question 5

Task 1: Construct a Sankey Diagram

```
nodes <- data.frame(
  name = c("Residential", "Suburban", "Rural",
           "Bus", "Train",
           "Central", "Industrial")
```

```
)  
  
links1 <- data.frame(  
  source = match(data$Origin, nodes$name) - 1,  
  target = match(data$Mode, nodes$name) - 1,  
  value = data$Commuters  
)  
  
links2 <- data.frame(  
  source = match(data$Mode, nodes$name) - 1,  
  target = match(data$Destination, nodes$name) - 1,  
  value = data$Commuters  
)  
  
links <- rbind(links1, links2)  
  
sankeyNetwork(  
  Links = links,  
  Nodes = nodes,  
  Source = "source",  
  Target = "target",  
  Value = "value",  
  NodeID = "name",  
  fontSize = 12,  
  nodeWidth = 30  
)
```



Task 2: Construct a Parallel Sets Representation

Convert data into parallel sets format

```
parallel_data <- gather_set_data(
  data,
  c("Origin", "Mode", "Destination"),
  id_name = "FlowID"
)
```

Create Parallel Sets diagram

```
ggplot(
  parallel_data,
  aes(
    x = x,
    id = FlowID,
    split = y,
    value = Commuters
  )
) +
```

```

geom_parallel_sets(aes(fill = Origin), alpha = 0.8) +

geom_parallel_sets_axes(axis.width = 0.1) +

geom_parallel_sets_labels(colour = "black") +

labs(

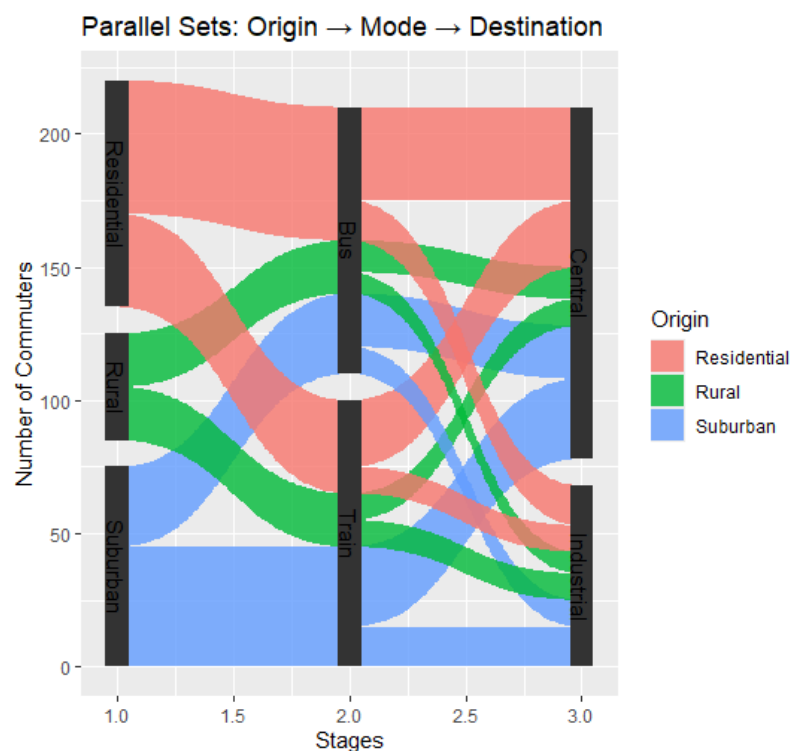
  title = "Parallel Sets: Origin → Mode → Destination",

  x = "Stages",

  y = "Number of Commuters"

)

```



Task 3: Identify the Three Largest Flows and Their Percentages

```

largest_flows <- data[order(-data$Commuters), ][1:3, ]

largest_flows$Percentage <-

  (largest_flows$Commuters / sum(data$Commuters)) * 100

print(largest_flows)

```

Expected Result

Origin	Mode	Destination	Commuters	Percentage
Residential	Bus	Central	35	17.5%
Suburban	Train	Central	30	15%
Residential Train Central			25	12.5%

Task 4: Compare Sankey and Parallel Sets

Sankey Diagram	Parallel Sets
Shows movement as flowing links	Shows categorical relationships across axes
Link width represents commuters	Band width represents commuters
Best for understanding dominant flows	Best for comparing categories across multiple stages
Easy to trace movement	Easy to see relationships between categorical variables

Conclusion:

Sankey is better for clearly seeing the magnitude and direction of flows. Parallel sets are better for understanding relationships between Origin, Mode, and Destination categories.

Task 5: How Can Poor Link Width or Category Ordering Mislead?

Answer:

If link widths are not proportional to the number of commuters, a small flow may appear more important than a large flow.

Also, if categories are arranged poorly, many flow lines may cross each other, making it difficult to follow the actual flow and possibly misleading the viewer.

